

Behavioral Anomaly Detection Report

Behavioral Anomaly Detection for Cybersecurity

Technical Report

Problem Statement 4A — AI-Powered Behavioral Anomaly Detection for Cybersecurity

Submitted by Anshaditya Sharma

1. Problem and approach

Model “normal” access behaviour per entity (user, service account, edge device), detect intrusions and compromised-credential activity from access logs, classify the anomaly type, and produce a risk score an analyst can act on — under extreme class imbalance and with no real intrusion data to train on.

The approach in one paragraph: generate a synthetic access log with injected attacks and ground-truth labels; turn each session into ~22 behavioural features, two of which are z-scored against a trailing per-entity window so legitimate behaviour change stops being flagged; score every session with three models (isolation forest, random forest, GRU) blended into one risk score; pull new entities’ scores toward a population prior until they have history of their own; explain the top 5% of alerts with SHAP composed into a sentence; and let an analyst confirm or dismiss alerts to re-rank the queue.

Every number in this report comes from the artifacts in `data/processed/` and `reports/figures/` produced by `python -m src.pipeline` and `python -m src.demo.coldstart_drift` at seed 42.

Headline result: PR-AUC 0.899 for binary anomaly detection, and 353 of the 354 sessions in a top-1% alert budget are genuine attacks. **Headline caveat:** the system’s *classifier* and its *risk score* disagree about what matters — device spoofing sessions are correctly typed 99.4% of the time but only 1.8% of them rank into that top-1% queue. Section 4 covers this; it is the most important limitation in the system and it is structural, not a tuning problem.

2. Synthetic data generator

Real access logs for this problem are scarce and privacy-restricted, so `src/generator/` builds one: 250 users, 40 service accounts, and 80 edge devices, each with its own habitual login hours, home location, typical resource set, device fingerprint, and (for privileged entities) command vocabulary, sampled with noise over a 45-day window. A random 7% of entities “join” partway through the window so the cold-start path is exercised by construction rather than by accident.

Seven labelled patterns are injected on top of normal traffic — six genuine attacks and one deliberately ambiguous edge case:

Pattern	How it is simulated
Brute force	12-60 rapid failed logins from one attacker IP against one entity
Impossible travel	A session placed 0.5-3h after a real one, from a location implying an impossible travel speed
Credential stuffing	20-70 entities, 1-3 attacker IPs, high failure rate, all inside a short window
Lateral movement	A burst of 6-16 sessions touching never-before-accessed resources, often with privileged commands
Device spoofing	Same entity_id, mismatched OS/MAC fingerprint
Low-and-slow exfiltration	1-3 sessions on scattered days across a 10-25 day span (most days skipped), concentrated in the entity’s off-hours window, gradually working through unseen resources
Insider drift	A legitimate entity gradually expanding its resource footprint over 1-3 weeks — not a hard positive; used to measure false-positive behaviour on slow, ambiguous change

Low-and-slow exfiltration and insider drift are built to look similar on purpose: both are gradual, both accumulate new resources over weeks. They differ in that exfiltration concentrates in off-hours while drift samples from the entity's own normal hour distribution, so drift keeps looking routine. Whether the system separates them is a real test rather than a given, and section 4 reports the answer.

Resulting dataset:

Label	Sessions	Share of log
normal	109,305	96.71%
brute_force	791	0.700%
insider_drift	710	0.628%
credential_stuffing	576	0.510%
low_and_slow_exfil	443	0.392%
impossible_travel	437	0.387%
lateral_movement	432	0.382%
device_spoofing	327	0.289%
total	113,021	3.29% anomalous

Per-type injection rates land between 0.29% and 0.70%; combined anomaly rate is 3.29%, marginally above the brief's suggested 0.5–3% band. Ground truth lives in a separate `labels.csv` and is joined back only for training and evaluation — the access log itself carries no label column, matching what would be available at inference time.

Known simplifications: geo coordinates are jittered around a home point rather than drawn from a real road/flight network, so impossible-travel distances are geometrically but not geographically realistic. Command sequences are drawn from a fixed action vocabulary rather than modelling real shell/API semantics.

3. Models

3.1 Features

`src/features/build_features.py` turns each raw session into 22 features: time-of-day and day-of-week, hours since the entity's last session, geo distance and implied velocity from both the previous session and from home, whether the resource or device fingerprint has ever been seen for this entity, rolling failure counts and distinct-entity counts per source IP, and rolling resource breadth per entity.

Two design choices matter more than the rest:

- **Resource breadth is computed at two window sizes.** `entity_resource_breadth_24h` catches a burst of new resources in one sitting (lateral movement's signature). `entity_resource_breadth_7d` rolls over a full week, so an accumulation of one or two new resources a day across many days registers even though no single day looks unusual. The 7-day window exists specifically because low-and-slow exfiltration is constructed to be invisible to the 24-hour one.
- **Session duration and login hour are z-scored against a trailing 30-day per-entity window**, not all-time history (`src/features/drift.py`). This is the concept-drift handling; section 6 measures it.

3.2 The three models

- **Baseline profiler** (`models/baseline.py`) — an isolation forest per `entity_type` plus a per-entity statistical profile used by the dashboard’s entity-history view.
- **Tabular classifier** (`models/classifier.py`) — a random forest over the same features, multi-class over `normal` plus the seven injected labels, `class_weight="balanced"`, `n_estimators=300`, `max_depth=20` (section 5 explains that depth). Chosen partly because it is SHAP-friendly via `TreeExplainer`, so it doubles as the model behind the explanation layer.
- **Sequence model** (`models/sequence_model.py`) — a single-layer GRU over each entity’s last 10 sessions, multi-class, trained with inverse-frequency class weights. This is the sequence-aware component: it can pick up patterns that only exist across a run of sessions rather than in any single row.

Risk score is a fixed blend:

```
raw_risk = 0.25*isolation_forest + 0.35*rf_anomaly_prob + 0.40*gru_anomaly_prob
```

then pulled toward the `entity_type`’s mean risk for cold-start entities (`models/cold_start.py`), weighted by how little history exists. Predicted anomaly type is the argmax of the averaged random-forest and GRU class probabilities.

The blend weights are hand-set, not learned. They were not tuned against the test split, which is the honest version of “we did not optimise them” rather than a claim that they are optimal.

4. Evaluation

Split by time, not randomly: the first 70% of the simulation window trains, the last 30% tests, so the sequence model and rolling features are never evaluated on data that leaked into their own history.

Test window: 35,494 sessions, 839 anomalous, of which 645 are genuine attacks (the remaining 194 are `insider_drift`, excluded from the binary target).

4.1 Headline metrics

Metric	Value
PR-AUC, binary anomaly vs. normal	0.899
Precision @ top 1% alert budget	0.997
False positive rate @ top 1% alert budget	0.003
Overall multi-class accuracy	0.960
Macro-average F1	0.749
Weighted-average F1	0.973

The gap between macro F1 (0.749) and weighted F1 (0.973) is the story of this system in two numbers: it is excellent on the classes that dominate by count and poor on two of the seven.

4.2 Per-class results

Class	Precision	Recall	F1	Support
impossible_travel	0.985	1.000	0.992	130
normal	0.999	0.962	0.980	34,655
brute_force	0.953	0.988	0.970	165
credential_stuffing	0.906	0.975	0.939	79
low_and_slow_exfil	0.953	0.762	0.847	80
lateral_movement	0.535	0.895	0.670	76
device_spoofing	0.246	0.983	0.394	115
insider_drift	0.118	0.665	0.201	194

- **Impossible travel, brute force, credential stuffing** are close to solved. Their signatures (geo-velocity, rapid failed auths, one IP against many accounts) are almost definitional given the feature set.
- **Low-and-slow exfiltration works.** 0.953 precision and 0.847 F1 mean the 7-day breadth window plus the off-hours signal genuinely separate it from both lateral movement (fast breadth) and insider drift (normal-hours breadth). This was the open question when the class was added, and the answer is yes.
- **Lateral movement is mediocre** (0.535 precision, high recall) — it over-flags.
- **Device spoofing and insider drift are the weak classes.** Insider drift is *expected* to be weak: it is defined as not-clearly-anomalous, which is why it is excluded from the binary target. Device spoofing has no such excuse, and section 5 investigates it.

4.3 What the alert budget buys

Alert budget	Sessions surfaced	Genuine attacks	insider_drift	normal	Precision
top 1%	354	353	0	1	0.997
top 2%	709	572	78	59	0.917
top 5%	1,774	625	151	998	0.437
top 10%	3,549	641	186	2,722	0.233

There are only 645 genuine attack sessions in the test window, so past roughly the top 2% the queue exhausts the real attacks and fills with normal traffic and insider drift. The system is calibrated for a tight budget and should not be presented as usable at a loose one.

Across the full log, the top-1% queue (1,131 sessions, cutoff 0.886) contains **zero normal sessions and zero insider_drift sessions**, and all 1,131 are labelled with exactly the type the model predicted. At this operating point the queue is clean.

4.4 The typing/ranking divergence

The cleanliness of that queue hides the system's real weakness. Correctly *classifying* a session and *ranking* it highly are different things, and for half the attack types they come apart badly:

Class	Correctly typed	In top-1% by risk score
brute_force	99.7%	90.1%
lateral_movement	98.1%	21.3%
low_and_slow_exfil	95.7%	3.6%
device_spoofing	99.4%	1.8%

The model knows what a device-spoofing session is — it labels 99.4% of them correctly — but the ensemble risk score puts almost none of them in front of an analyst. Same for low-and-slow exfiltration, which is arguably by design: a pattern built to avoid single-session spikes will not produce a single session with a spiking score. Its sessions cluster at mean risk 0.809 (std 0.076) against a top-1% cutoff of 0.886, so 4% clear it; widen to top 2% and 42% clear it, at top 5% 98% do.

This is a scoring problem, not a detection problem. The information needed to surface these sessions already exists inside the model and is being discarded by a fixed 0.25/0.35/0.40 blend that was never tuned. A per-class score calibration, or simply routing high-confidence type predictions into the queue regardless of blended risk, would likely fix it. Neither is implemented, and claiming the top-1% precision number without this caveat would be misleading.

5. The device_spoofing capacity investigation

Device spoofing precision was 0.70 in the 7-class version of this system and fell to 0.20 when `low_and_slow_exfil` was added as an 8th class. The first hypothesis was that a fixed-capacity forest (`max_depth=10`) had run out of room to separate one more class. Rather than write that into the report as an accepted limitation, it was tested.

Method. Refit only the tabular classifier at six capacity settings, reusing the shipped baseline profiler and GRU unchanged, and re-score the same test split. The `depth=20, n=300` row reproduces the shipped numbers exactly, which is the self-consistency check that makes the rest of the table trustworthy.

Config	device_spoofing	lateral_movement	insider_drift	low_and_slow_exfil	accuracy
depth=10, n=300	0.199	0.479	0.078	0.822	0.937
depth=15, n=300	0.222	0.544	0.098	0.938	0.950
depth=20, n=300 (shipped)	0.246	0.535	0.118	0.953	0.960
depth=None, n=300	0.256	0.543	0.163	0.953	0.971
depth=15, n=500	0.222	0.548	0.096	0.938	0.950
depth=20, n=500	0.249	0.548	0.119	0.953	0.960

(precision per class; `n` is `n_estimators`)

Result: capacity helped, but did not fix it. Device spoofing improves from 0.199 to 0.256 across the sweep — a 29% relative gain, not nothing — but it remains the weakest class by a wide margin at every setting tested. No capacity configuration brings it anywhere near usable. Capacity was a real contributing factor and a wrong diagnosis at the same time.

What is actually wrong. The confusion breakdown on the test window, for the 459 sessions predicted as device_spoofing:

True label	Count	Share of that class predicted device_spoofing
normal	312	0.9%
device_spoofing	113	98.3%
insider_drift	25	12.9%
low_and_slow_exfil	8	10.0%
credential_stuffing	1	1.3%
brute_force / impossible_travel / lateral_movement	0	0.0%

Two things at once. In *absolute* terms most false positives are normal sessions (312 of 346), simply because normal is 34,655 sessions. But in *rate* terms the two gradual classes are 10-14x more likely to be mislabelled device spoofing than normal traffic is, and the three sharp attack classes are never confused with it at all. Device spoofing's only strong signal is a binary fingerprint mismatch; when that bit is ambiguous the forest falls back on generic novelty features, which is exactly the region of feature space the two gradual classes occupy. It has become the model's catch-all for "novel, but not in a way I recognise."

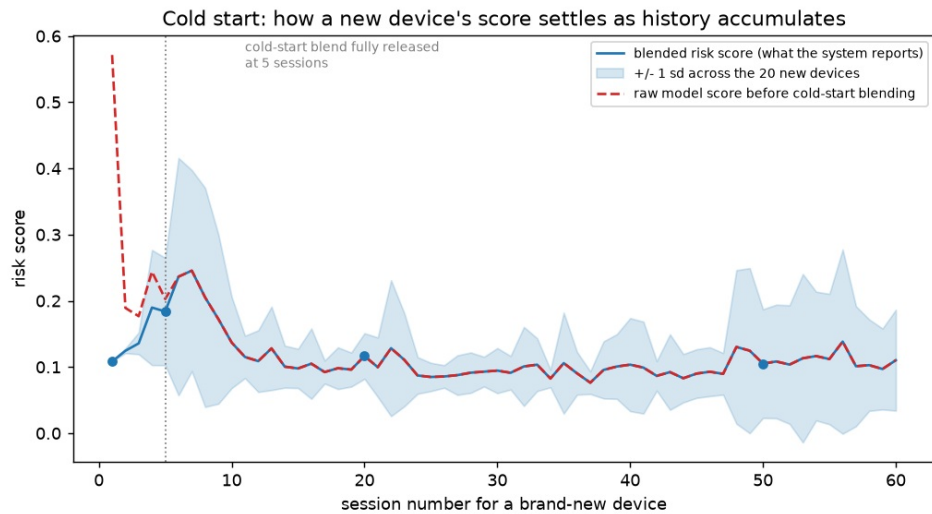
Fixing this requires a feature that distinguishes a genuine fingerprint change from other kinds of novelty — fingerprint change *rate* per entity, or fingerprint stability over a trailing window — not more trees. That is not implemented.

Why depth=20 and not depth=None. On this split, unbounded depth is better on essentially every metric, including the class this investigation was about. It was not adopted, and the reason is a judgement call rather than a measurement: unbounded trees are the most overfit-prone configuration tested, and there is only one time split in this project to check against. Adopting the best number on the only split you have is how you end up reporting a result that does not survive a reseed. That reasoning is defensible but it is not evidence, and a second holdout period would settle it properly.

6. Cold start and concept drift

Both are named evaluation criteria, so rather than assert they work, `python -m src.demo.coldstart_drift` reproduces the evidence below and writes the two charts into `reports/figures/`. The dashboard's "System behaviour" tab renders the same charts and numbers.

6.1 Cold start



Cold start: how a new device's score settles as history accumulates

Twenty brand-new edge devices, none present at training time, scored at increasing history depth:

Session	Reported risk	Raw model risk	sd across devices	Weight on own history
1	0.108	0.571	0.000	0.00
5	0.184	0.203	0.081	0.80
20	0.117	0.117	0.034	1.00
50	0.105	0.105	0.082	1.00

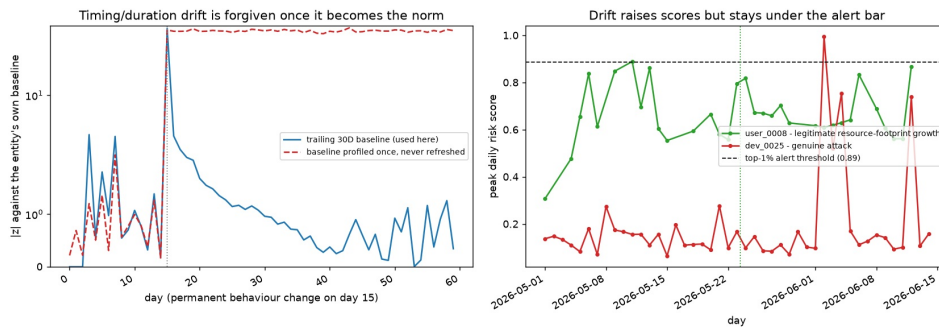
The first row is the point. A device on session one scores **0.571** from the models alone — it is novel on every feature that exists: unseen resource, unseen fingerprint, no prior session to compare against. That is a false positive waiting to happen. Blending against the population prior reports **0.108** instead, far below the 0.886 top-1% cutoff.

The mechanism is a linear ramp: $w \cdot \text{own_score} + (1-w) \cdot \text{prior}$, where $w = \min(1, \text{history}/5)$. An entity stops being cold after 5 sessions, and the prior is fixed at training time so scoring a new entity does not depend on whatever else happens to be in the batch.

Two honest caveats:

- **The confidence band is misleading at exactly the point it looks best.** Spread across devices is 0.000 at session 1 — not because the system is confident, but because every new device receives the identical prior. Real disagreement only appears once own-history takes over (sd 0.034 at session 20), and it *widens* rather than settles (sd 0.082 by session 50) as devices diverge onto their own profiles. The first-session score is the least informative one despite looking the most certain.
- **Five sessions is a count, not a duration.** A chatty edge device clears cold start in under an hour; a service account that authenticates weekly stays cold for over a month; and a genuinely compromised new account has its score suppressed for its first five sessions. The threshold should be a function of expected session rate per entity type. It is not.

6.2 Concept drift



Timing/duration drift is forgiven; resource-footprint drift is not

What works (left panel). On a synthetic series whose behaviour changes permanently on day 15, the trailing 30-day z-score spikes to $|z|$ **50.70** when the change happens — correctly flagging it — then settles to $|z|$ **0.58** once the new behaviour is simply the norm. A baseline profiled once and never refreshed sits at $|z|$ **47.82** indefinitely: it would flag that entity forever. That is precisely the failure mode the rolling window exists to avoid, and it is avoided.

What does not work (right panel). The rolling window only covers timing and duration. Resource-footprint growth is scored by `is_new_resource`, computed against all-time first-seen, and nothing ever forgives a permanently expanded resource set. Across the 34 insider-drift entities with enough history to compare:

- median risk rose from **0.203** before the shift to **0.714** during it
- it rose for **34 of 34** entities — there is no favourable case to point at
- even the most favourable entity rose, 0.533 to 0.672

So: drift handling is real for *when* and *how long* an entity works, and entirely absent for *what it touches*.

What keeps this from being an operational problem at the current operating point is that drifting entities do not clear the bar — **0 of 710** insider-drift sessions reach the top 1%. They raise the score without raising an alert. The margin is thin, though: at a top-2% budget 66 of them appear, and at top 5%, 623 do. The fix (decaying resource novelty, or scoring novelty against a trailing window like the timing features) is not implemented.

7. Explainability

`src/explain/attribution.py` runs SHAP's `TreeExplainer` against the random forest for the top 5% of sessions by risk score — 5,652 sessions — on the grounds that nobody needs a reason for a session nobody will look at. The top 3 SHAP features are composed into one sentence by `src/explain/narrative.py`. Real output from the current run:

Flagged as brute force due to 27 failed logins for this entity within 10 minutes, combined with a 100% authentication failure rate from this source IP.

Flagged as credential stuffing due to a 95% authentication failure rate from this source IP, combined with a source IP being used against 18 other accounts in the same 10-minute window and a device fingerprint that does not match this entity's prior sessions (now reporting Windows10).

Flagged as impossible travel due to a login 16,632 km from this entity's usual location, combined with an implied travel speed of 185,812 km/h since the previous session.

This is deterministic templating, not a generation call, for two reasons: the same alert must read identically every time an analyst opens it, and it runs for every alert in the queue rather than once for a demo. Raw SHAP values stay available behind an expander in the dashboard so the sentence can be checked rather than trusted.

Where it degrades honestly. SHAP attributes positively to features whose *values* are unremarkable, because trees split on low values too. Early versions produced “a 0% authentication failure rate” and “an implied travel speed of 1 km/h” as grounds for an alert, which is worse than silence. Each phrase now renders only above a value worth reading (geo-velocity >100 km/h, IP failure rate >20%, $|z| > 1.5$, and so on). These are display thresholds only — they do not affect any score.

The consequence is that when no feature clears its threshold, the alert says so rather than inventing a reason:

Flagged as low-and-slow exfiltration on a combination of weak signals, with no single feature standing out — see the SHAP breakdown.

That fallback fires for the two classes whose signature is distributed across many sessions rather than concentrated in one — low-and-slow exfiltration and lateral movement. It is the correct behaviour for a single-session explainer, and it is also an admission that a per-session explanation is the wrong unit for a multi-session attack. An entity-level explanation (“this account touched 9 new resources over 12 days, all outside its normal hours”) would serve those two classes far better and does not exist.

Also worth stating: SHAP runs against the random forest only, not the GRU. The two models can disagree on the predicted type, and the displayed reason is strictly faithful only to the forest’s view.

8. Analyst feedback loop

The dashboard records confirm/dismiss decisions per alert into a local SQLite file and converts them into a per-entity adjustment applied only to queue ranking. This is deliberately not learning and should not be described as such: it does not retrain anything, does not modify stored risk scores, does not generalise from one entity to another, and never forgets. It is a manual re-ranking knob with an audit trail.

Two capping modes exist in `src/dashboard/feedback.py`. The original capped the raw score offset; the current default caps movement in percentile rank. Both are kept so the comparison below runs on the same data.

All numbers in this section hold the alert cutoff fixed at its original value (0.886) rather than recomputing it after adjustment, so they isolate an entity’s own movement rather than mixing in a shift of the whole distribution.

8.1 The original score cap bounded almost nothing

The first design was **−0.02 per dismissal, +0.02 per confirmation, capped at ±0.10** of raw risk score. That cap turned out to be nearly decorative. Across the 213 entities with at least one alert in the top-1% queue, the margin between an entity’s highest-scoring alert and the queue cutoff distributes like this:

Margin above cutoff (score)	Value
median	0.030
mean	0.033
75th percentile	0.044
maximum	0.114

The ± 0.10 cap is **larger than the margin for 204 of 213 entities (95.8%)**. Only nine entities score high enough to survive five dismissals. At session level, 1,002 of the 1,131 queued alerts (88.6%) could be pushed below the cutoff, including 798 genuine attack sessions (70.6%).

The root cause is that a fixed *score* offset does not correspond to a fixed amount of *queue movement*. Risk scores cluster tightly just above the cutoff instead of spreading out above it, so 0.10 of score spans most of the queue. The cap's value had no interpretable relationship to what it was supposed to bound — nobody chose “88.6% of the queue”, it just fell out of the score distribution.

This is an insider-threat surface, not a usability quirk. An analyst who is themselves the threat, or whose account is compromised, could suppress detection of their own activity through the intended UI, leaving an audit trail that looks like ordinary triage.

8.2 Replacing it with a percentile-rank cap

The fix caps how far an entity can move in **percentile rank** rather than in score. Requested movement is `net_decisions * 0.05` percentile points, clipped to $\pm$ `PERCENTILE_CAP` however many decisions were recorded, then mapped back through the empirical score distribution so the queue can still rank on a score. Per-decision step times max steps still equals the cap ($0.05 \times 5 = 0.25$), so “five decisions maxes it out” is preserved — only the ceiling changes units.

Why this unit is the right one. The top-1% queue spans one percentile point (99.0002 to 100.0000). A cap of N percentile points therefore means precisely “an entity can be moved through N% of the queue”, independent of how tightly scores happen to cluster. The measured relationship is exactly linear, which is the property the score cap lacked:

Percentile cap	Entities fully removable	Queue sessions removable
0.05	6 / 213 (2.8%)	57 / 1,131 (5.0%)
0.10	16 / 213 (7.5%)	114 / 1,131 (10.1%)
0.15	31 / 213 (14.6%)	170 / 1,131 (15.0%)
0.20	42 / 213 (19.7%)	227 / 1,131 (20.1%)
0.25 (shipped)	60 / 213 (28.2%)	283 / 1,131 (25.1%)
0.30	74 / 213 (34.7%)	340 / 1,131 (30.1%)
0.50	153 / 213 (71.8%)	566 / 1,131 (50.0%)
1.00	213 / 213 (100%)	1,131 / 1,131 (100%)

Justifying N = 0.25. The original ± 0.10 was justified only as “five dismissals $\times$ 0.02”, a number with no operational meaning. N is justified against the tradeoff curve above. $N \geq 1.0$ is equivalent to no cap at all, since the queue is one percentile point deep. $N = 0.50$ still leaves 72% of entities removable, barely better than before. $N = 0.10$ is tighter but lets an analyst move an entity only 10% of the way down the queue, which is thin for genuine triage. $N = 0.25$ keeps a quarter of the queue's depth available for legitimate correction (~282 positions in the full log) while cutting entity exploitability from 95.8% to 28.2%. That is a policy choice on a smooth continuum rather than a discovered optimum — but it is now a *legible* choice, which the previous parameter was not.

8.3 Before and after, measured

Measure	Score cap ± 0.10	Percentile cap 0.25
Entities fully removable from queue	204 / 213 (95.8%)	60 / 213 (28.2%)
Queue sessions pushable below cutoff	1,002 / 1,131 (88.6%)	283 / 1,131 (25.1%)
Genuine attack sessions suppressible	798 / 1,131 (70.6%)	87 / 1,131 (7.7%)

The set of entities still exploitable under the new cap is a strict subset of those exploitable under the old one — nothing became newly vulnerable.

The `dev_0025` insider-threat case, re-run. This is the entity with the largest queue footprint: 54 alerts in the top-1% queue and 55 genuine brute-force sessions in its history.

	Best rank after 5 dismissals	Queue alerts remaining
No dismissals	12	54 / 54
Score cap ± 0.10	937	37 / 54
Percentile cap 0.25	248	54 / 54

Under the new cap it is properly bounded: all 54 of its alerts stay in the queue, where the old cap silently removed 17 of them. Note the rank still moves a long way (12 $\rightarrow$ 248) — the entity is deprioritised, which is what an analyst dismissing it should achieve, but it cannot be made to disappear.

(The previous version of this report quoted “54 alerts to 40” for the old cap. That figure recomputed the 1% cutoff on the adjusted distribution; holding the cutoff fixed, as everything here does, the correct figure is 37.)

8.4 What this does not fix

60 entities are still fully removable, and they are the wrong 60. Every one of them is a genuine attacker, and they are almost entirely low-footprint: of the 60, most have one or two alerts in the queue. The residual surface is concentrated on entities whose attack is *small* — predominantly `credential_stuffing`, plus `device_spoofing`, `lateral_movement`, `low_and_slow_exfil` and `impossible_travel` cases with a single qualifying session.

That is arguably backwards from what you want. The percentile cap protects high-volume, high-scoring attackers well — precisely the ones an analyst is least likely to miss anyway — and protects the quiet ones worst. A patient attacker who keeps their footprint to one or two sessions is still fully suppressible by five clicks. The change reduces the insider-threat surface by roughly 9x in genuine-attack terms (798 $\rightarrow$ 87 sessions), which is a real improvement, but it does not close it, and what remains is skewed toward subtle attacks rather than obvious ones.

Legitimate correction is now more limited too, and this is a real cost. 144 entities that an analyst could previously clear out of the queue entirely can no longer be cleared. For a genuinely noisy high-scoring entity — a misconfigured scanner that trips brute-force detection every day — the analyst can now only push it 25% down the queue and must keep seeing it. 848 of 1,131 queued sessions can still be moved down but not out. Whether that trade is correct depends on how much you trust the analyst, which is exactly the question the cap exists to avoid having to answer.

Everything else about the loop is unchanged. The cap bounds how far one entity can move; it does nothing about *who* is moving it. Still not implemented: flagging the meta-pattern of one analyst repeatedly dismissing alerts on one entity, requiring a second reviewer above some risk

floor, decaying offsets over time, or detecting that a dismissal contradicted a known label. The percentile cap makes the blast radius of a single malicious analyst smaller and predictable; it does not detect one.

8.5 Other limits of the loop

- **It starts empty.** With zero decisions every offset is zero, so the loop contributes nothing on day one and only becomes useful after an analyst has worked the queue for a while.
- **It is per-entity, so it does not transfer.** Dismissing a noisy scanner teaches the system nothing about the next identical scanner.
- **Decisions never expire.** An entity dismissed five times in January is still suppressed in June — the feedback loop has a concept-drift problem of its own, which is a slightly embarrassing thing for this particular system to be carrying.

9. Known limitations

Consolidated; the sections above give the evidence for each.

Detection and scoring

- The risk score and the type classifier disagree for half the attack types (§4.4). Device spoofing is typed correctly 99.4% of the time and reaches the top-1% queue 1.8% of the time. Until the blend is calibrated per class, the top-1% precision figure describes a queue dominated by fast, spiky attacks and says little about slow ones.
- Device spoofing precision (0.246) is a feature-space problem, not a capacity problem (§5). It needs a fingerprint-stability feature that does not exist.
- Ensemble blend weights (0.25/0.35/0.40), isolation-forest contamination, and GRU class weights are hand-set constants, never tuned.
- `max_depth=None` outperformed the shipped `max_depth=20` on every tracked metric on this split and was rejected on overfitting grounds that were reasoned about but not measured (§5).

Evaluation validity

- One seed, one time split. Direction of every result is clear-cut; exact numbers would move on a reseed, and nothing here has been checked against a second holdout period.
- Insider drift is deliberately excluded from the binary anomaly target, so PR-AUC describes performance on genuine attacks only.
- The synthetic generator encodes the same assumptions the detector exploits. Impossible travel is easy to catch partly because the generator draws attacker locations uniformly at random. Performance here is an upper bound on performance against a real adversary.

Cold start and drift

- The cold-start threshold is a session count, not a duration, so it treats a chatty device and a weekly service account identically (§6.1).
- A genuinely compromised new entity has its score suppressed for its first five sessions — the blend cannot distinguish “new and unfamiliar” from “new and hostile”.
- Drift is forgiven for timing and duration, never for resource footprint (§6.2).

Explainability

- Explanations are per-session, which is the wrong unit for multi-session attacks; those correctly fall back to “no single feature stood out” (§7).
- SHAP runs against the random forest only, so the reason shown is faithful to that model rather than to the ensemble that produced the score.
- Narrative display thresholds were chosen by inspection, not derived from the data.

Feedback loop

- The abuse cap is now expressed in percentile rank (0.25 points = 25% of the queue) rather than raw score, which cut entity exploitability from 95.8% to 28.2% and genuine-attack suppressibility from 70.6% to 7.7% (§8.3). It is a reduction, not a fix.
- The residual surface is skewed toward *quiet* attackers: all 60 still-removable entities are genuine attacks, and most have only one or two alerts in the queue (§8.4). A patient attacker with a small footprint remains fully suppressible by five clicks.
- The new cap also constrains legitimate use — 144 entities that an analyst could previously clear out of the queue no longer can (§8.4).
- The cap bounds blast radius but detects nothing: no flagging of one analyst repeatedly dismissing one entity, no second-reviewer requirement, no offset decay, and no detection of a dismissal that contradicts a known label.
- Offsets remain per-entity and never expire.

10. Scalability and real-time feasibility

Measured on the full 113,021-session log, single process, commodity laptop CPU:

Stage	Wall time	Per session	Throughput
Feature construction	4.07 s	36.0 μ s	~27,800/s
Scoring (all three models)	4.73 s	41.9 μ s	~23,900/s
End-to-end	8.81 s	77.9 μs	~12,800/s

At ~12,800 sessions/sec end-to-end, a single process handles roughly 1.1 billion sessions/day of throughput headroom, so raw compute is not the constraint for any plausible deployment. Two things are:

State, not compute. Feature construction is dominated by per-entity and per-source-IP rolling windows. Batch mode recomputes these over the whole log; a streaming implementation would instead keep per-entity state (last session timestamp and location, running failure counts, a 7-day resource set) and per-IP state (10-minute session and failure counters). That is naturally incremental and cheap per event, but it means the operational problem is a state store sized by *entity and IP cardinality*, not by event volume — and it must survive restarts, since a lost per-entity baseline puts every entity back into cold start. The 30-day drift window and 7-day breadth window set the retention floor.

Training and explanation, not inference. Inference is 42 μ s/session; retraining the ensemble takes minutes and SHAP attribution over the top 5% is the slowest stage in the pipeline by a wide margin. This is why attribution is capped at 5% of sessions rather than run over everything. A production deployment would score in-stream and retrain on a schedule, which is the usual shape and is compatible with this design.

What has not been tested. Everything above is single-process batch throughput on one seed's data. There is no streaming implementation, no measurement of state-store size or restart behaviour, and no latency distribution under concurrent load — only aggregate throughput. Treating these numbers as evidence that a real-time deployment is *feasible* is fair; treating them as evidence that it would *work* is not.